Unidad 1 / Escenario 1

Lectura fundamental

Arquitectura del *software*

Contenido

1 ¿Qué es la arquitectura?

2 Arquitectura vs el desarrollo de *software*

3 Requerimientos y su relación con la arquitectura de *software*

4 ¿Qué son los patrones de arquitectura?

Palabras clave: drivers, vistas, patrones, arquitecto, framework.

1. ¿Qué es la arquitectura?

Existen numerosas definiciones de arquitectura en múltiples campos (médico, civil, hidráulico, militar, naval, religioso, etc.) y, específicamente, para el campo de la ingeniería. En este módulo se propone manejar una definición buscando acertar en tres aspectos: simplicidad, precisión y completitud.

Así pues, arquitectura es el arte y técnica de diseñar, proyectar, definir y construir elementos y relaciones entre ellos.

Se citan a continuación dos referentes, considerados los más relevantes y acertados en la definición:

> Según la Real Academia de la Lengua Española: **"Del lat. architectūra. f. Inform. Estructura lógica y física de los componentes de una computadora"** (RAE, 2018).
>
> Según Len Bass, Paul Clements y Rick Kazman (2012) en su libro Software Architecture in Practice: **"La arquitectura de software de un sistema es el conjunto de estructuras necesarias para razonar sobre el sistema, que comprende elementos de software, relaciones entre ellos y propiedades de ambos.**" (citado en: Cervantes Maceda, Velasco-Elizondo y Castro Careaga, 2016)

A continuación, se realiza una precisión sobre las palabras utilizadas en la primera definición en el contexto de la ingeniería de *software*:

Arte y técnica: es una forma de expresión fruto de la experiencia o inspiración, que permite exponer asertivamente lo que se quiere transmitir. Esto se plasma a través de técnicas (mecanismos, estándares) especializados y consensuados, lo que logra eliminar ambigüedades, manejar un mismo concepto, lenguaje o estilo.

Diseñar: es la acción mediante la cual se materializa una idea a través de una vista, modelo, dibujo, plano o esquema.

Construir: este trabajo consiste en determinar la mejor forma de configurar **elementos y relaciones entre ellos**; dentro de estos elementos se encuentran piezas de código, servicios, módulos, clases, métodos, capas y entidades.

Proyectar: es una habilidad necesaria en la elaboración de los diseños; corresponde a una visión del producto final o *software* que se necesita fabricar. Esto permite que, en la etapa de construcción, todas las condiciones, funcionalidades y requerimientos de calidad del *software* sean tenidos en cuenta.

Definir: quizás es uno de las acciones más olvidadas. La arquitectura no depende únicamente del esquema, dibujo o plano. Se refiere a realizar una especificación de las características de los elementos de *software* o del lenguaje utilizado (gráfico y no gráfico) y la forma como debe ser abordado por distintos roles o puntos de vista.

1.1. Vistas y puntos de vista

Una vista de arquitectura es un producto de trabajo que expresa la arquitectura de un sistema desde una perspectiva de una preocupación-problema-interés (concern) específico (IEEE, 2011). Por ejemplo, un diagrama de secuencia presenta una solución frente al interés de modelar la interacción entre objetos en un sistema.

Un punto de vista de la arquitectura es un producto de trabajo que establece las convenciones para la construcción, interpretación y uso de vistas de arquitectura correspondiente a una preocupación-problema- interés (concern) específico en un sistema (IEEE, 2011). Por ejemplo, la ficha de descripción de un caso de uso.

1.2. El rol del arquitecto

En la ingeniería de *software* tenemos distintos roles; cada uno de ellos maneja lenguajes de comunicación o modelos para entender o diseñar la parte del *software* de la cual se encargan. Por ejemplo, un analista de bases de datos entiende muy bien un modelo relacional o clave valor; sin embargo, para un desarrollador front-end es más familiar una paleta de colores, que poco tiene que ver con el trabajo del analista.

Un arquitecto de *software* debe enfrentarse a distintos retos, entre ellos, ser mediador de los lenguajes y disciplinas anteriormente mencionadas; además de eso, lidiar con las constantes nuevas tecnologías y, al mismo tiempo, con las preocupaciones-problemas- intereses (concerns) de un sistema.

Los concerns o drivers de arquitectura son expresados y capturados por la interacción con distintos roles (jefes, arquitectos, programadores, diseñadores, etc.). Este trabajo debe estar enfocado principalmente en cumplir los requerimientos de calidad de la aplicación o sistema. Para lograr esto debe estar desarrollando constantemente habilidades duras y blandas.

1.3. Estilos

¿Sabía qué...?

Según el portal de contratación y estadística salarial glassdoor (https://www.glassdoor.com), un arquitecto de *software* gana entre 97 mil dólares y 169 mil dólares al año, aproximadamente 40 millones de pesos al mes. Un solo arquitecto gana aproximadamente lo que 54 personas ganarían si trabajan con el salario mínimo colombiano. No todo en la vida es dinero, pero cómo ayuda...

Siempre que se requiera diseñar o construir una arquitectura, seguramente alguien más se ha tenido que enfrentar al mismo problema o a uno similar; por esto, es importante considerar alternativas de solución existentes. En ese sentido, dos referentes comúnmente usados en la arquitectura de *software* son los patrones y los estilos. El estilo de arquitectura es la manera en que los arquitectos resuelven una situación de diseño y comprende un nivel de abstracción mayor que los patrones. Los patrones son soluciones puntuales a problemas (de estructura, comportamiento y creación) recurrentes en el desarrollo de *software*, mientras que los estilos resuelven requerimientos de calidad del sistema; su enfoque holístico o nivel de abstracción mayor no se debe confundir con las Arquitecturas de Referencia (estas diferencias se tratarán más adelante). Los estilos también pueden corresponder a la forma en que se documenta una vista. De acuerdo con Clements et al. (2011), en ese caso existen:

- *Styles of the Module Viewtype*
- *Styles of the C&C Viewtype*
- *Styles of the Allocation Viewtype*

Estos estilos pueden tener subestilos, pero este tema no está dentro del alcance de este módulo.

A pesar de esto, cuando se habla de estilos arquitectónicos se hace referencia a la formulación o expresión de un esquema de organización que es fundamental para los sistemas de *software* y rara vez hace alusión a la forma en que se documentan.

El estilo arquitectónico establece un conjunto predeterminado de tipos de elementos, especifica sus responsabilidades e incluye reglas y guías para organizar las relaciones entre ellos (Rozanski y Woods, 2005). Algunos ejemplos de los estilos más utilizados son:

- Cliente/servidor
- Capas
- Filtros y tuberías
- N-Tercios
- Par a par
- Publicador-suscriptor.

1.4. Frameworks

Un framework es un conjunto reutilizable de bibliotecas o clases de *software*. Constituye una herramienta para el desarrollador que le ayuda a enfocar su trabajo en tareas más complejas y ahorrar tiempo de codificación.

Un patrón es una solución concreta, un código o estructura de código que sirve para resolver una situación particular (adicional al código existen un conjunto de elementos debidamente documentados que serán explicados más adelante). En el proceso de construcción de un framework se requiere tener en cuenta, entre otros aspectos, los patrones. Los frameworks tienen una estructura modular y también se enfocan en la resolución de un problema específico en un sentido más amplio que los patrones; son soluciones concretas que deben estar construidas teniendo en cuenta los patrones. Ejemplos de las soluciones o problemas en los que se centra el framework son: la seguridad, construcción de interfaces tanto locales como web, la comunicación remota, la persistencia de objetos en una base de datos relacional y un amplio etcétera.

En la actualidad existen muchos frameworks (ver figura 1) enfocados en resolver una gran diversidad de problemas recurrentes en la construcción de sistemas de *software*. La figura muestra uno de varios criterios que deben ser tenidos en cuenta a la hora de escoger el framework; es importante mencionar que un criterio no es suficiente para determinar su uso, en otras palabras, solo la popularidad no es una medida de la efectividad de la herramienta. La selección del framework debe contemplar los requerimientos funcionales y no funcionales del *software* más que una estadística, que, para el caso, puede estar influenciada por la facilidad de uso, aprendizaje, el tiempo que lleva en el mercado, etc.

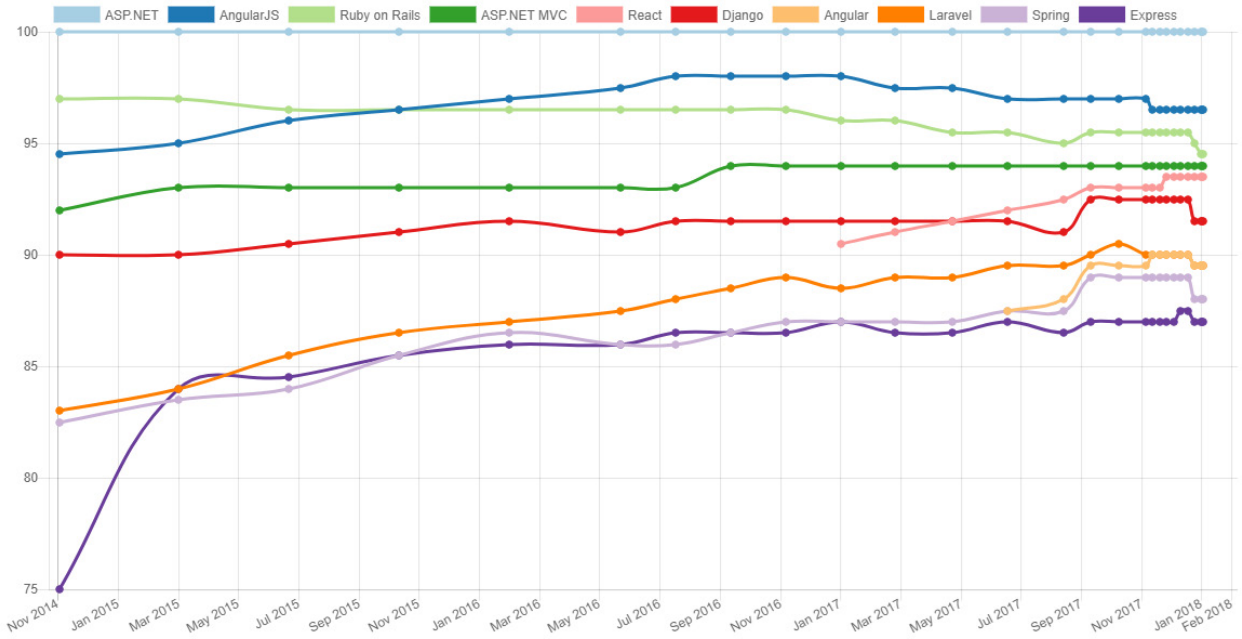

Figura 1. Medida de la popularidad promedio por Github, Stack Overflow y Overall

Fuente: Hotframeworks (s.f.)

El uso de frameworks puede ocasionar varios inconvenientes, por ejemplo, dependencia, una acentuada curva de aprendizaje, la dificultad de elección dada la gran cantidad de productos, no conseguir el personal calificado para su uso, consecución de portales que den estadísticas fiables, la evolución constante del producto y su rápida obsolescencia, gran cantidad de código que no se necesita (sobre todo en aplicaciones pequeñas), la falta de soporte y los costos de licencia. A pesar de todos esos inconvenientes, hoy en día es difícil imaginar un diseño de un sistema sin el uso de un *framework*.

Tabla 1. Top 14. Media de la popularidad promedio por Github, Stack Overflow y Overall

Framework	Github Score	Stack Overflow Score	Overall Score	Framework	Github Score	Stack Overflow Score	Overall Score
ASP.NET		100	100	Laravel	91	88	89
AngularJS	96	97	96	Spring	84	92	88
Ruby on Rails	91	98	94	Express	91	83	87
ASP.NET MVC		94	94	Meteor	92	80	86
React	100	87	93	Vue.js	99	74	86
Django	89	94	91	CodeIgniter	82	86	84
Angular	90	89	89	Symfony	83	85	84

Fuente: Elaboración propia (2018). Modificado de Hotframework (s.f.)

Cómo mejorar...

Utilice múltiples criterios de evaluación, ajústelos ponderándolos mediante una medida cuantitativa según los requerimientos o drivers del sistema que se esté construyendo.

Buenos referentes son empresas multinacionales de consultoría, universidades y escuelas de ingeniería.

Busque sitios que estén en continua actualización y que no tengan compromisos comerciales.

2. Arquitectura vs. el desarrollo de *software*

Si bien la arquitectura define, acompaña, estructura y proyecta gran parte del desarrollo de *software*, este implica muchas más cosas. La arquitectura de *software* tiene repercusión directa sobre las primeras fases de desarrollo de *software* mientras que, en el caso de las pruebas, el despliegue y post mortem solamente tiene influencia o una relación poco marcada.

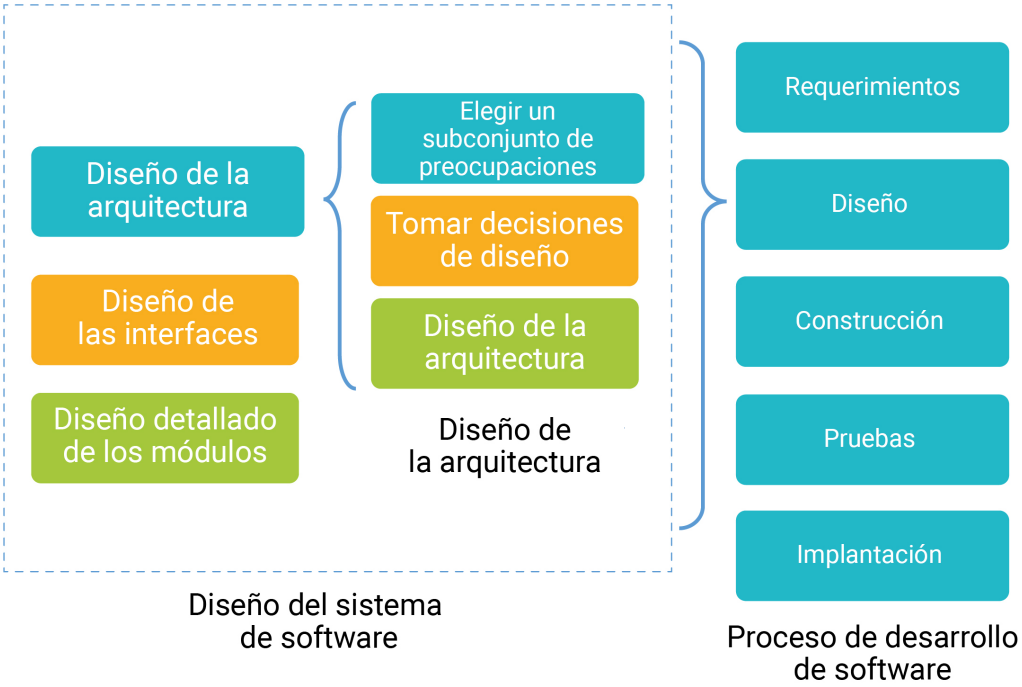

Figura 2. Arquitectura vs. el proceso de desarrollo de software

Fuente: Elaboración propia (2018). Modificado de Cervantes Maceda, Velasco-Elizondo y Castro Careaga (2016)

Independiente del tipo de metodología (ej. RUP, SCRUM, XP), las fases presentadas en la figura 2, que corresponden al proceso de desarrollo de *software*, pueden cambiar de duración, al igual que la forma como se iteran o recorren, pero se conservará esta estructura base presentada.

3. Requerimientos y su relación con la arquitectura de *software*

Los requerimientos de *software* son el insumo más importante para realizar arquitectura de *software*. Esto contempla las distintas categorías, desde los requerimientos de negocio, pasando por los funcionales y no funcionales y, finalmente, los técnicos.

La arquitectura de *software* se centra en atender, en su mayoría, los requerimientos no funcionales que son los relacionados con la calidad del sistema. Algunos de estos pueden ser escalabilidad, interoperabilidad, desempeño, usabilidad y seguridad. Como clave, la mayoría de ellos terminan en "dad".

Figura 3. Tipos de requerimientos

Fuente: Elaboración propia (2017). Modificado de Dorfman (1997)

3.1. Drivers vs. requerimientos

Los drivers o motivadores son los requerimientos que tienen mayor relevancia para el diseño de la arquitectura. Independientemente del tipo de requerimiento, el desarrollo y diseño de la arquitectura en algunos casos será nulo o escaso y en otros de gran aporte, por lo que no todos deben ser tenidos en cuenta para conformar la estructura arquitectónica. A continuación, se presenta una clasificación de los distintos tipos de drivers en los cuales pueden ser clasificados y ordenados los requerimientos del sistema.

3.2. Drivers arquitectónicos

El tiempo de para realizar el diseño de *software* es escaso (Cervantes Maceda, Velasco-Elizondo y Castro Careaga, 2016). El proceso de selección y priorización de los requerimientos de *software* genera los drivers de arquitectura, los cuales, adicionalmente, pueden ser ordenados en las siguientes categorías:

- Drivers de restricciones: todas las restricciones del sistema deben ser tenidas en cuenta en el diseño de la arquitectura y, a diferencia de los demás tipos de drivers, esas no pueden ser priorizados o deben tener el mismo nivel de importancia.
- Drivers funcionales: como se mencionó en la definición de drivers de arquitectura, corresponden a un subconjunto de los requerimientos, en este caso de los funcionales, que pueden ser priorizados según el impacto que puedan ejercer en la arquitectura del sistema, los objetivos del negocio que afectan y la complejidad técnica de implementación.
- Drivers de reglas de negocio: es de vital importancia que el diseño se ajuste a las necesidades del entorno, ya que no todas las decisiones de diseño de la arquitectura son de tipo técnico.
- Drivers de componentes externos: las soluciones de arquitectura no son aisladas, por lo que gran parte del diseño debe tener en cuenta las interacciones con otros sistemas. Estos deben ser considerados con alta prioridad en el desarrollo de la arquitectura, junto con los sistemas o componentes legados.
- Drivers de atributos de calidad: por lo general, todos los requerimientos de software no funcionales están asociados a la calidad del sistema y, por ende, todos son drivers de calidad.

3.3. Documentación de drivers y requerimientos de *software*

Usualmente se suele confundir el nivel de detalle con el nivel de abstracción del requerimiento. El nivel de detalle está asociado con la cantidad de información y la claridad del requerimiento, mientras que el nivel de abstracción se relaciona con el tipo de requerimiento. A continuación, se presentan algunas fichas de requerimientos o drivers con un nivel de detalle medio para cada uno.

El arquitecto debe determinar, dependiendo del grado de relevancia, cuál debe ser el nivel de detalle; claramente, para el grupo de arquitectura, los drivers deberán tener mayor detalle que los requerimientos generales.

Documentación de requerimiento o driver de negocio:

Es el nivel más alto de abstracción; usualmente se elabora o documenta en compañía de los directivos o dueños de negocio, junta directiva o administrativa.

Tabla 2. Ejemplo de documentación de requerimiento funcional (detalle medio)

ID requerimiento	Requerimiento de negocio	Responsable	Obligatorio/ Altamente deseable / deseable	Proceso de negocio	Objetivo(s) de negocio relacionados
RN_001	El Inventario de almacén debe ser actualizado cuando se reciba un nuevo producto.	Administrador de almacén	Obligatorio	Recibir inventario	Realizar seguimiento de manera oportuna a los productos.

Fuente: elaboración propia

Documentación de requerimiento funcional

Son todas las operaciones de cara al cliente o usuario final; describen, en su mayoría, el comportamiento de la interface de usuario ante los distintos eventos ejecutados por él. Como una función, describe una entrada, un proceso y una salida.

Tabla 3. Ejemplo de documentación de requerimiento funcional (detalle medio)

Id requerimiento	Nombre	Descripción o proceso	Entradas	Salidas
RF_001	Asignar aspectos generales de la categorización.	Listar tipos de taxón (especie, subespecie, variedad, población) Seleccionar tipo de evaluación (global, regional, nacional) Seleccionar la duración de la generación.	Evento listar taxón Evento listar tipo de evaluación Duración de la generación.	Asignar aspectos generales de la categorización Registro en base de datos.

Fuente: elaboración propia

Documentación de requerimiento no funcional drivers de calidad:

Definen características de calidad del sistema; gran parte de estos requerimientos conforman los drivers de arquitectura, específicamente los drivers de calidad.

Tabla 4. Ejemplo de documentación de requerimiento no funcional drivers de calidad (detalle medio)

Id requerimiento	Nombre	Descripción
RNF_001	Rendimiento	El número (#) de terminales: de 100 terminales a 30 mil (estimando que el mayor porcentaje son colombianas) # esperado de usuarios simultáneamente conectados: menos de 100 # de transacciones por segundo que deberá soportar el sistema: 10 El 95 % de las transacciones deben realizarse en menos de 1 segundo Se cuentan con alrededor de 2.227.000 registros biológicos y 3700 fichas.

Fuente: Elaboración propia (2018)

Técnicos:

Tabla 5. Ejemplo de requisitos técnicos de hardware (detalle medio)

Requisitos técnicos de hardware	
Descripción general:	RTH_001 Los requerimientos mínimos de hardware con los que debe contar una máquina cliente para operar adecuadamente el sistema de inventarios.
Elementos	
Procesador	2,5GHz
Memoria	8Gb o superior
Pantalla	Resolución mínima 2048*968 o superior
Tarjeta de red	Ethernet 10/100/1000/ e inalámbrica 802,11
Disco duro	1T
Otros	Puerto USB 3,0

Fuente: elaboración propia

Tabla 6. Ejemplo de requisitos técnicos de software (detalle medio)

Requisitos técnicos de *software*	
Descripción general:	RTS_001 El *software* base con el que debe contar una máquina cliente para operar adecuadamente el sistema de inventarios.
Elementos	
Browser	Chrome v52 o superior
Actualizaciones	Sistema operativo Windows xp: Service pack 4 Sistema operativo Vista: todas las actualizaciones de seguridad Sistema operativo Windows 8 y 10: Service pack 2 2. Todos los equipos deben tener instalado el siguiente driver: http://suppot.Microsoft.com/kb/54545454
Otros	Conexión con certificado digital *Driver del certificado digital expedido por "alguna marca" Certificado digital: validado por "alguna marca"

Fuente: elaboración propia

4. ¿Qué son los patrones de arquitectura?

En términos generales, un patrón es una solución recurrente a un problema específico de *software*. Los elementos esenciales para que un patrón sea considerado como tal son: nombre fácil de recordar en inglés, definición, problema específico recurrente y solución: pseudocódigo, diagrama de clases, patrones relacionados o participantes.

Algunas definiciones formales de patrón de *software* son:

"Un patrón de diseño es una descripción de clases y objetos comunicándose entre sí adaptada para resolver un problema de diseño general en un contexto particular" (Gamma, Helm, Johnson & Vlissides, 1994)

"Cada patrón describe un problema que ocurre una y otra vez en un determinado contexto y describe una solución al problema de tal manera que esta solución puede ser utilizada millones de veces de forma recurrente" (Alexander, Ishikawa & Murray, 1980)

Un patrón es un "modelo que sirve de muestra para sacar otra cosa igual" (RAE, 2018); se centra en la problemática de la forma y el contexto, derivado de las ciencias exactas, matemáticas y lógica. Se trata de desglosar los problemas complejos de diseño.

Por el tipo de enfoque o aplicación, existen distintos tipos de agrupación de los patrones:

4.1. GOF (Gang of Four)

Design Patterns: Elements of Reusable Object-Oriented Software es un libro de ingeniería de *software* que describe patrones de diseño de este. Esos patrones se denominan GOF debido a los 4 autores del libro, referenciados a menudo como "la banda de los 4": Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, con un prólogo de Grady Booch. El libro está dividido en dos partes; los dos primeros capítulos son acerca de las capacidades y dificultades de la programación orientada a objetos, y los capítulos restantes describen 23 patrones clásicos de diseño de *software*. El libro incluye ejemplos en C ++ y Smalltalk (Gamma et al., 1994).

4.2. GRASP (General Responsibility Assignment Software Patterns)

En diseño orientado a objetos, object-oriented design, GRASP, son patrones generales de *software* para asignación de responsabilidades; sin embargo, en este módulo NO serán considerados propiamente patrones ya que no cumplen con todos los elementos esenciales, como, por ejemplo, el pseudocódigo de implementación y el problema asociado es demasiado general. Una nominación más adecuada para GRASP es principios de diseño o características que deben ser consideradas a la hora de diseñar o codificar. Los GRASP no son soluciones concretas a problemas funcionales; por el contrario, se asocian más a requerimientos generales de calidad como, por ejemplo, el patrón HIGH COMPLEXITY cuyo objetivo es reducir la característica de complejidad (ver Larman, 2004).

4.3. Concurrencia

Si bien la concurrencia se podría observar como un solo patrón, la complejidad y la técnica alrededor de su implementación ameritan una clasificación particular. Dentro de las características más importantes a considerar encontramos el acceso al servicio y configuración, manejo de eventos, concurrencia y sincronización. Esta clasificación cobra alto valor en sistemas que involucran redes de comunicación o en la arquitectura orientada a servicios. Un libro centrado en concurrencia y *networking es Pattern-Oriented Software Architecture, Patterns for Concurrent and Networked Objects* (Schmidt, Stal, Rohnert y Buschmann, 2000).

4.4. Fowler

Para Martin Fowler un patrón es una idea que ha sido útil en un contexto práctico y probablemente será usado en otros (Fowler, 1997). Si bien estos patrones describen más una arquitectura a nivel empresarial, o dominios de negocio, constituyen una fuente importante para la arquitectura de *software*, toda vez que permiten definir rápidamente el funcionamiento empresarial a través de sus procesos. En su libro Analysis Patterns: Reusable Object Models, Fowler proporciona un catálogo de patrones que han surgido en una amplia gama de dominios, incluidas las relaciones comerciales, de medición, contables y organizacionales.

Algunos de estos patones se han empleado en su libro en forma combinada para obtener otros modelos; este análisis está basado en la forma como las personas perciben el mundo (Fowler, 1997). Algunos de los patrones son: Party, Organization Hierarchies, Quantity, Accountability, Conversion Ratio, entre muchos otros.

En síntesis...

El estudio de la arquitectura de *software* representa un gran trabajo. Más que una tarea épica, es una historia llena de grandes experiencias formalizadas a través de la documentación disciplinada; contiene entre otras, las mejores configuraciones y estrategias en el proceso de construcción y diseño de *software*.

Este interesante mundo requiere de gran disposición para aprovechar la experiencia en cada uno de los campos y poder determinar los aportes de valor. Este escenario consolida los aspectos más relevantes en el mundo de la arquitectura de *software*.

Referencias

Alexander, C., Ishikawa, S. & Murray, S. (1980). *A Pattern Language Towns-Buildings-Construction*.

Bass, L., Clements, P. & Kazman, R. (2012). *Software Architecture in Practice*. New Jersey. United States: Addison-Wesley.

Cervantes Maceda, H., Velasco-Elizondo, P. y Castro Careaga, I. (2016). *Arquitectura de software, conceptos y ciclo de desarrollo.* Mexico: CENGAGE Learnig.

Clements P., Bachman, F., Bass, L., Garlan, D., Ivers, J., Little, R., ...Stafford, J. (2011). *Documenting Software Architectures, Views and beyond*. Adisson-Wesley Professional.

Dorfman, M. (1997). *Requirements Engineering.* En R. Thayer y M. Dorfman (Eds.), Software Requirements Engineering (pp. 7-22). Fort Collins, EUA: Computer Society Press.

Fowler, M. (1997). *Analys Patterns: Reusable Object Models*. USA: Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R. & Vlissides, j. (1994). *Design Patterns Elements of Reusable Object-Oriented Software*. EUA: Addison Wesley.

Hotframeworks. (s.f.). *Find your new favorite web framework*. Recuperado de https://hotframeworks.com/

IEEE. (2011). *Systems and software engineering. Architecture description.* Suiza: ISO/IEC- IEEE. Recuperado de http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6129467&isnumber=6129466

Larman, G. (2004). *Applying UML and Patterns. An Introduction to Object-Oriented Analysis and Design and the Unified Process*. EUA: Addison Wesley Profesional.

RAE. (2018). *Real Academia de la lengua Española.* Recuperado de http://www.rae.es/

Rozanski, N. y Woods, E. (2005). *Software Systems Architecture*. EUA: Addison-Wesley.

Schmidt, D., Stal, M., Rohnert, H. y Buschmann, F. (2000). *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects.* Chichester, England: Jhon Wiley & Sons.

Referencias de figuras

Hotframeworks. (s.f.). *Medida de la popularidad promedio por Github, Stack Overflow y Overall* [Gráfico]. Recuperado de https://hotframeworks.com/

INFORMACIÓN TÉCNICA

FACULTAD DE
INGENIERÍA, DISEÑO
E INNOVACIÓN

Módulo: Arquitectura del software

Unidad 1: Introducción y conceptos generales sobre arquitecturas del software

Escenario 1: Arquitectura del software

Autor: Diego Iván Oliveros Acosta

Asesor Pedagógico: Jeiner Velandia

Diseñador Gráfico: Alejandro Torres

Asistente: Leidy Alejandra Morales Eslava

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.